

로그 RCA 를 위한 증거 보존형 KV Cache 압축 기법

위치(lost-in-the-middle)·롱테일 원인 보존을 겨냥한 로그 전용 KV 압축기
— 최종 통합본 (도구 정당성 + 선행연구 근거 + 실행 계획) —

2026

프로젝트 한 문장

긴 로그를 KV 캐시로 압축할 때, 로그 고유 신호(레벨·희귀도·시간·위치)로 '희귀한 원인 라인'과 '중간에 묻힌 증거'를 보존하는 하이브리드 압축기를 만들고, 일반 압축법보다 원인을 더 잘 보존함을 실험으로 보인다.

1. 배경 — 왜 이 문제인가

로그 분석의 4대 태스크(파싱·이상탐지·RCA·요약) 중 RCA만 유독 미성숙하다. 프런티어 모델조차 OpenRCA 벤치마크에서 완전 정답률이 3.9~12.5%에 그친다. 실패의 상당 부분은 모델의 절대 능력이 아니라 시스템 구조에서 기인하며(환각 71%), RCA를 어렵게 만드는 핵심 요인 중 하나가 '긴 로그'다.

장애 케이스당 로그는 수천~수만 줄, 약 100K 토큰에 달한다. 원인 라인은 방대한 정상 로그에 묻히고, 모델은 긴 문맥에서 중간 정보를 놓치는 lost-in-the-middle을 겪는다. 동시에 긴 문맥은 KV 캐시를 폭발시켜 GPU 메모리 병목을 만든다.

KV 병목의 정량 근거: LLaMA-3.1-8B에서 128K 토큰 KV 캐시는 약 17.2GB로 모델 파라미터 메모리(약 16GB)를 넘어선다. 컨텍스트를 1K→120K로 늘리면 8B 모델 처리량이 약 77배 급감한다. (FastKV §2.1, Cartridges §1)

핵심 딜레마: 캐시는 줄여야 하는데(메모리), 막 줄이면 원인이 사라진다(정확도). 어떻게 줄이면서 원인은 보존하는가?

2. 왜 필터가 아니라 KV 압축인가 — 인트로 논리 사슬

이 프로젝트의 정당성은 두 관문을 통과해야 성립한다. **(2단계)** 왜 입력 필터가 아니라 KV 층위에서 압축하는가, 그리고 **(3단계)** 왜 기존 일반 KV 압축이 아니라 로그 특화 압축이 필요한가.

2.1 (2단계) 입력 필터로 대체되지 않는 이유

노이즈 제거만 놓고 보면 입력 필터(ERROR/WARN·키워드·정규식)가 KV 압축보다 싸고 간단하다. **이 점은 인정한다.** 따라서 KV의 존재 이유를 '노이즈 제거'에 두지 않는다. 주장은 좁고 구체적이다 — 필터가 원리적으로 못 하는 것이 있고, 로그 RCA는 바로 그것이 중요한 도메인이다. **실제로 현업 RCA 파이프라인**은 이미 프롬프트 단계에서 필터링을 쓴다(RCACopilot은 warning/error만 입력, LogSage는 ERROR-only 필터링이 맥락을 누락시킨다고 경고). 즉 필터는 기본값이고, 문제는 '필터만으로 충분한다'다.

① 필터의 메모리 절감은 '보수적 하한'에 막힌다

필터도 토큰을 줄이므로 KV 메모리를 줄이기는 한다(토큰 수에 KV가 선형 비례). 문제는 **얼마나 내려갈 수 있는가**다. 이 프로젝트의 전제 자체가 '원인은 종종 평범한 INFO, 튀지 않는 줄'이므로, 필터는 원인을 날릴까 봐 **보수적으로만** 걸어낼 수 있다. 그 결과 '안전하게 관련 있는 집합'이 여전히 100K 토큰급으로 남는다. **필터의 안전 하한이 KV 메모리 예산보다 크다** — 필터는 여기서 더 못 내려가고(내려가면 원인 유실), KV 압축이 그 잔여를 head·layer 단위로 회수한다. 즉 둘은 경쟁이 아니라 계층이다.

② head·layer 단위 세밀 보존 — 필터는 줄을 통째로만 버린다

필터는 로그 줄을 넣거나 빼는 이진 결정만 가능하다. '이 줄은 서비스 연결 판단엔 필요하지만 시간 추론엔 불필요'라는 구분을 못 한다. KV 압축은 한 토큰을 어떤 attention head에서는 버리고 다른 head에서는 유지할 수 있다. (주의: '입력으로 만들 수 없는 새 캐시 상태 생성'이라는 더 강한 표현력 주장은 학습 기반 KV(Cartridges 계열)의 성질이라 우리가 쓰는 eviction엔 적용되지 않는다. eviction에 유효한 것은 head 단위 보존까지다.)

③ lost-in-the-middle 완화는 KV 내부에서만 가능하다

입력 프루닝(필터)은 버려진 토큰의 hidden state가 이후 레이어로 전파되지 않아 비가역적 정보 손실을 일으킨다(전역 문맥이 필요한 작업에서 성능 급락). 반면 KV 방식은 초기 레이어가 전체 시퀀스를 처리한 뒤 선택·전파하므로 전체 문맥 근사를 유지한다(FastKV). 우리가 겨냥한 lost-in-the-middle 완화는 필터로는 원리적으로 도달할 수 없다. (이 이점을 살리려면 압축이 초기 레이어의 전체-시퀀스 처리 후에 적용돼야 한다 → §6 설계에 반영.)

정직한 경계 (과대주장 방지): 표현력·head 제어 근거 중 일부는 학습 기반 KV의 우월성이지 eviction(H2O·SnapKV류)의 우월성을 곧바로 보장하진 않는다. eviction은 압축률이 높아지면 성능이 급락한다는 보고도 있다. 따라서 방어 가능한 입장은 'eviction이 필터보다 항상 낫다'가 아니라,

'필터가 원리적으로 못 하는 영역이 로그 RCA에 필요하고, 로그 고유 신호로 그 영역에서 eviction을 개선한다'는 것이다.

직렬 파이프라인 프레이밍: 필터와 KV는 경쟁 도구가 아니라 서로 다른 병목을 공략하는 직렬 파이프라인이다. 필터=진입 전 명백한 노이즈 제거(입력 토큰), KV=필터 후에도 남는 GPU 메모리 병목을 head 단위로 회수. 지도 피드백의 '두 개를 모두 구현해야 할 수도 있다'가 정확히 이 구조다.

2단계 반증 조건 (§9 보강)

- 필터만으로 GPU 메모리가 목표치 안에 들어오면 → KV의 여지 없음(직렬 파이프라인 전제 붕괴 신호)
- 필터 후 남는 로그에서도 eviction이 필터 대비 원인 보존에 이점이 없으면 → head 단위 보존 가설 기각

2단계 근거 출처

- head 단위 제어·표현력 우위: Cartridges (Stanford, arXiv:2506.06266) §B.3, §B.3.2 — 단 학습 기반 KV에 한정
- 입력 프루닝의 비가역적 정보 손실·전역 문맥 붕괴: FastKV (arXiv:2502.01068)
- 하드-토큰(필터) 압축이 KV 대비 성능 열위: End-to-End Context Compression (arXiv:2606.09659) — 2026-06 프리프린트, 미심사
- 현업 입력 필터링 관행: RCACopilot(LogPilot 인용) · LogSage(2506.03691) · RCAgent(CIKM 2024)

2.2 (3단계) 일반 KV 압축으로 부족한 이유 — 근거 강화

일반 KV 압축은 attention 기반(H2O·SnapKV·Scissorhands)과 최근성/싱크 기반(StreamingLLM)으로 나뉜다. 두 계열 모두 RCA가 의존하는 정보를 구조적으로 버린다.

① attention 점수는 중요도의 손실 많은 대리지표.

- H2O (NeurIPS 2023): 누적 attention 높은 'heavy hitter'만 남김 → 희귀 원인 라인은 attention을 못 받아 제거 후보.
- RazorAttention (ICLR 2025): 중요도 기반 드롭이 '주제와 덜 관련된 정보 질의 시 작동 안 함'을 실증. 정상 로그 대비 '벗어난' 원인 라인이 이 케이스.
- Scissorhands (NeurIPS 2023): '한번 안 중요하면 계속 안 중요' 가정 → 질의 순간에야 결정적이 되는 원인 라인에서 깨짐.

② 최근성/싱크 기반은 설계상 중간을 버림.

- StreamingLLM (ICLR 2024): 초기 싱크 + 최근 창만 남기고 중간 삭제. 싱크는 의미가 아니라 수학적 이유로 선택됨.
- Lost in the Middle (TACL 2024): U자형 성능, 중간 배치 시 문서를 안 준 경우(56.1%)보다 낮을 수 있음. 시간 중간에 묻힌 원인 = 이중 처벌.

③ needle-in-haystack에서 압축법이 붕괴 (원인=바늘).

- 보호 없는 H2O는 NIAH 검색 F1이 거의 0(0.003, 전체 0.117 대비). DynamicKV는 StreamingLLM·PyramidKV 대비 최대 57% 개선 → 그 baseline들이 바늘을 크게 놓친다는 뜻.

④ 도메인 불일치 — 로그로 튜닝·평가된 적 없음.

- LongBench·RULER 등 QA·요약·추론으로만 평가. Pitfalls of KV Cache Compression(UCLA 2025)은 압축이 '선택적 기억상실'로 프롬프트 일부를 조용히 무시하게 만든다고 보고 — 희귀 로그 클래스를 조용히 버리는 것과 동형.

⑤ 중요 토큰은 태스크 의존 — 고정 휴리스틱 일반화 실패.

- KV 압축 성능은 태스크 의존적(Semantic Integrity Matters 2025). 로그는 고유 중요도 구조(희귀 템플릿·에러 레벨·시간 인과)를 가진 별개 태스크 → 자체 신호 필요.

3. 타겟 — 누구를 위한 기술인가

- 대규모 마이크로서비스를 운영하며 LLM으로 장애 RCA를 자동화하려는 SRE/DevOps 팀, AIOps 회사 (Datadog·Splunk 등)
- 고통: 긴 로그 → LLM RCA 비용 폭발 + 막 줄이면 원인 놓침 → 아직 사람이 봐야 함(human-in-the-loop)
- 수요 증거: Meta Top-5 42% back-testing, Datadog Bits AI SRE '보조' 프레이밍. 순수 자율 RCA 배포는 아직 부재

4. 핵심 결정 — 무엇을 겨냥하는가 (RCA 판단 축)

KV 캐시 압축과 상성이 좋은 축(위치·롱테일)을 주력으로, 상성이 나쁜 축(멀티홉)은 배제한다.

RCA 판단 축	우리 겨냥	KV 압축과 상성
위치 (lost-in-the-middle)	✅ 핵심 주력	좋은 — 노이즈 제거로 중간 증거 부각
롱테일 (희귀 원인)	✅ 핵심 주력	좋은 — 드문 것 보존이 자연스러움
단일홉 원인 구조	✅ 겨냥	좋은 — 원인 줄 하나 보존
멀티홉 원인 구조	⌚ 확장 과제	⚠️ 나쁨 — 사슬 끊기 위험
멀티모달	❌ 배제	해당 없음 — 로그만 사용
라인 단위 평가	✅ 확정	—

멀티홉을 뺀 이유: 모델 안에서 KV를 버리면 원인 사슬 연결고리가 끊겨 재추론이 막힌다. 멀티홉은 입력 샘플링(Gleaner) 영역이고, KV 압축의 강점은 위치·롱테일이다.

5. 선행연구 대비 위치

- **Gleaner (2026):** 트레이스 스마트 샘플링이 RCA를 42~107% 향상. 단 '전통 RCA 알고리즘'이 뒷단이라 노이즈 제거 효과가 큼.
- **H2O·SnapKV·StreamingLLM·PyramidKV:** KV 압축이지만 QA/추론 도메인. 로그 특화 아님(\$2.2 근거).
- **Lost in the Middle (TACL 2024):** 긴 문맥에서 중간 정보를 놓친다는 것을 실증. 우리가 완화하려는 문제의 근거.
- **LLM4Log 서베이 (2026):** 컨텍스트 한계를 배포 난제로 지목, '가장 진단적 이벤트는 드물고 노이즈에 묻힌다'고 관찰. 단 KV 해법은 제시 안 함.

novelty(공백): 2026년 중반 기준, 로그·RCA·AIOps 특화 KV 캐시 압축 연구는 확인되지 않는다. '일반 KV 압축(로그 무관)'과 '로그 라인 우선순위화(프롬프트 단계)'가 따로 존재할 뿐, 이를 잇는 '로그 신호 기반 KV 압축기'는 미개척이다. (주의: LogQuant의 'Log'는 로그 데이터가 아니라 logarithmic 분포를 뜻하므로 선행연구로 오인 금지.)

6. 범위 경계 · 압축기 구현 틀

범위 — '압축'이지 '재사용'/'샘플링'이 아님

	우리 (KV 압축)	입력 샘플링(Gleaner)	KV 재사용(CacheBlend)
하는 일	모델 안에서 KV 버림	모델 밖에서 안 넣기	KV를 다른 위치에 붙임
목적	GPU 메모리 회수	저장·분석 비용 절감	재계산 시간 절감
강점 축	위치·롱테일	멀티홉·멀티모달	—
위치(RoPE) 문제	없음	없음	있음

스코프 — 하이브리드 (줄 판단 + KV 실행)

- 어떤 라인을 버릴지는 '줄 단위'로 판단
- 실제 제거는 그 줄의 토큰 KV를 삭제. 판단은 깨끗하게(줄), 실행은 진짜 KV
- 레이어 처리 방식(\$2.1③과 정합): 줄 keep/drop 판단은 전 레이어 공통으로 '한 번' 계산하되, 실제 압축은 초기 레이어가 전체 시퀀스를 통과한 뒤 적용한다(FastKV식). 이렇게 해야 lost-in-the-middle 완화 이점이 유지된다. '레이어 0부터 무조건 일괄 삭제'가 아님에 주의.

판단 신호 (라벨 없이 계산되는 로그 고유 신호)

- 로그 레벨: ERROR/WARN/INFO 등 이미 찍힌 중요도 힌트
- 희귀도: 드문 템플릿인가 / 반복 노이즈인가 (롱테일 겨냥)
- 시간·위치: 장애 시점에 튀었나 / 문맥 중간에 묻혔나 (lost-in-the-middle 겨냥)

압축 방식 — 가중 점수

- 각 줄에 여러 신호로 점수를 매겨 상위만 남김. 신호를 겹쳐 '드문 원인 vs 드문 무해' 구분
- 가중치·희색지대 경계는 예비 실험으로 조정 (틀은 확정, 세부는 하면서)

7. 핵심 난제 — 롱테일

진짜 원인은 대부분 '드문 로그'에 있다. 그런데 드문 것이 다 원인은 아니다. 빈도 하나만 보면 드문 원인과 드문 무해가 안 갈려 원인을 버린다.

'드물 = 진단적' 근거: Zebrium은 '처음 보는 이벤트는 대개 진단적'이라 보고(113개 중 2개만 신규이자 진단적), Red Hat Cordon은 진짜 이상을 '한두 번만 나타나는 의미상 고유한 이벤트'로 정의한다. AetherLog(ISSRE 2025)는 최근성 편향 때문에 LLM이 진짜 원인인 이전 시점 'nvme timeout'을 놓치는 실제 RCA 실패를 보인다 — 최근성 압축의 위험을 직접 증거.

그래서 여러 신호를 겹친다: '드물+ERROR+장애시점'이면 원인, '드물+INFO+정상시'면 무해. 완전 해결은 못 하며(RCA 근본 난제), 목표는 '빈도 단독보다 원인을 더 많이 남긴다'는 개선폭.

8. RCA 평가 방식 — 2층 구조

1층 (주력·안전판) — 압축률 대비 원인 보존율

- 같은 압축률(예: 80% 버림)에서 원인 라인을 누가 더 많이 남기나. 100% 통제 가능, 무조건 결과가 나옴
- 보존 정의: 원인 라인 KV가 전 레이어에서 살아있으면 1, 제거되면 0(이진). 여러 줄이면 M/N
- 라벨은 채점에만, 압축 시점엔 안 봄. baseline: 랜덤·최근성(StreamingLLM)·attention(H2O)·레벨 필터
- 결과물: 가로 압축률(50~90%), 세로 보존율 곡선. baseline보다 위면 우위

충화 필수: 전체 보존율은 레벨·희귀도 신호를 쓰는 순간 랜덤·최근성을 쉽게 이겨 무의미해 보일 수

있다. 반드시 (a) 전체 (b) 비-ERROR 원인 (c) 중간 위치 원인 서브셋으로 나눠 보고해야 프로젝트 동기가 된 하드 케이스에서의 우위를 증명한다.

2층 (검증·결정타) — RCA 정답률 차이(delta)

- 같은 압축률에서 압축 방식만 바꿔 뒷단 모델에 RCA. 절대값이 아니라 '우리 압축을 쓰면 더 잘 맞히나' 차이만 봄

주의(floor + 모델 확정): 보존율↑가 RCA↑를 보장하지 않는다(원본도 3.9%). 2층은 '향상 증명'이 아니라 '보존↔성능 관계 최초 측정'. 작은 오픈 모델은 자유서술 RCA에서 정답률이 바닥이라 delta 측정 자체가 불가하므로, ① 2층 태스크를 제약형(원인 후보 K개 선택/라인 지목)으로 설계해 floor를 띄우고, ② 2층 모델을 미리 확정한다(§10 참조).

9. 성공 기준 · 반증 조건

성공 기준

- 동일 압축률에서 우리 압축기의 원인 보존율이 baseline보다 통계적으로 유의하게 높음(특히 하드 케이스 서브셋)
- 구체 목표 수치는 예비 실험에서 baseline 성능을 측정한 뒤 확정

반증 조건 (끼워맞추기 방지)

- 레벨 필터만으로 보존율이 이미 99%면 → 우리 방법의 여지 없음(주제 재설정 신호)
- baseline과 보존율 차이가 통계적으로 없으면 → 로그 규칙성이 별 도움 안 됨
- 2층에서 RCA가 전혀 안 오르면 → 정보 보존이 병목이 아니었음
- 필터만으로 메모리 목표 달성 / 필터 후 eviction 이점 없음 → 도구 선택(§2.1) 전제 붕괴

10. 데이터 · 라벨링 · 모델

데이터셋

데이터셋	라벨 형태	용도	라벨링 부담
Thunderbird / BGL	라인 단위 이상(정상/비정상)	1차 시작	없음 — 이미 줄 라벨
Nezha	코드영역/구문 단위	확장(원인 보존)	⚠ 구문→줄 매핑 +검수

- 라벨은 압축기가 안 씀(채점용). 채점의 근간이라 정확도 중요 — 틀리면 결과 전체가 무너짐
- 라벨 누수 방지: 개발용/평가용(held-out) 분리. 튜닝은 개발용에서만

주장-데이터 정합성 경고: Thunderbird/BGL은 '이상' 라벨이지 '원인' 라벨이 아니다. 핵심 주장('원인은 종종 평범한 INFO')을 진짜 검증하는 건 원인 라벨이 있는 Nezha다. Nezha를 '여유 시 확장'이 아니라 **최소 축소판이라도 메인 실험 안으로** 당기길 권한다.

모델 선정 (권고)

용도	모델	근거
주력 backbone	Llama-3.1-8B-Instruct (128K)	KV 논문 최다 사용 · kvpress 지원 · baseline 직접 비교
2차 backbone	Qwen2.5-7B-Instruct (128K)	일반화 증명 · 추론 우수로 2층 floor 완화
baseline 도구	kvpress (NVIDIA)	H2O·SnapKV·StreamingLLM·PyramidKV 드롭인 재현
2층 floor 보조	DeepSeek-R1-Distill-Qwen-7B	추론 특화로 RCA floor 상승(선택)

인프라: 8B+100K는 단일 GPU로 충분. RunPod/Vast.ai 온디맨드(RTX 4090 ~\$0.34/hr, A100 ~\$1/hr)로 프로젝트 전체 대략 \$100~250. 안 쓸 땐 인스턴스 종료 필수.

11. 확정 · 미확정 요약

항목	정한 것	상태
겨냥 축	위치(lost-in-middle) + 롱테일 주력	✅ 확정
도구 정당성	필터 못 하는 3영역 → KV, 직렬 파이프라인	✅ 확정
스코프	하이브리드(줄 판단 + KV, 초기레이어 후 압축)	✅ 확정
보존 정의	이진(원인 라인 KV 살았나)	✅ 확정
압축 방식	가중 점수(레벨·희귀도·시간·위치)	✅ 확정
평가	보존율(충화) + RCA delta(제약형·2층)	✅ 확정
모델	Llama-3.1-8B + Qwen2.5-7B + kvpress	✅ 권고 확정
데이터	Thunderbird 1차 → Nezha 메인 당김 권고	✅ 확정
가중치 세부	각 신호 비중	⌚ 하먼서
예비 실험	레벨 필터 보존율 실측 (생사 관문)	⌚ 최우선
인력 배분	4명 역할	⌚ 미정

12. 6개월 흐름

- 1~2개월: Thunderbird 로드 + kvpress로 baseline(랜덤·최근성·H2O·레벨필터) 재현 + 레벨 필터 보존율 예비 실측 → 생사 관문 통과 확인
- 3~4개월: 라인→토큰 매핑 + 가중 점수 압축기 구현 + 압축률별 보존율 곡선(충화) 완성 → 핵심 결과
- 5~6개월: 2층 RCA delta(제약형) 측정 + Nezha 원인 보존(최소 축소판) + 발표 정리

한 문장 요약 긴 로그를 KV 캐시로 압축할 때 로그 고유 신호로 희귀한 원인 라인과 중간에 묻힌 증거를 보존하는 하이브리드 압축기를 만들고, 필터가 못 하는 3영역(메모리 하한·head 보존·LITM 완화)에서 KV를 정당화하며, 압축률 대비 보존율(충화)과 RCA delta(제약형)로 일반 압축 대비

우위를 검증한다.

부록. 논거별 참고문헌 매핑

범례 — [2단계] 왜 KV 증위인가 · [3단계] 왜 로그 특화인가 · [배경] 문제 정의

주장	핵심 근거(저자/연도/발표처)	단계
필터 못 하는 head 제어·표현력(학습 KV 한정)	Cartridges (Stanford, 2506.06266) §B.3	[2단계]
입력 프루닝의 비가역적 정보 손실·전역문맥 붕괴	FastKV (2502.01068)	[2단계]
하드-토큰(필터) 압축이 KV 대비 열위	End-to-End Context Compression (2606.09659, 미심사)	[2단계]
현업 입력 필터링 관행·맥락 누락	RCACopilot · LogSage(2506.03691) · RCAGENT(CIKM'24)	[2단계]
attention은 손실 많은 대리 지표 — 회귀 원인 제거	H2O(NeurlPS'23) · RazorAttention(ICLR'25) Scissorhands(NeurlPS'23)	[3단계]
최근성/싱크가 중간 문맥을 설계상 버림	StreamingLLM(ICLR'24) · Lost in the Middle(TACL'24)	[3단계]
needle-in-haystack에서 압축법 붕괴	DynamicKV(2412.14838) · 구조보호 연구(H2O F1 0.003)	[3단계]
QA·요약으로만 평가, 로그 미대상 (도메인 불일치)	Pitfalls of KV Cache Compression(UCLA 2510.00231)	[3단계]
중요 토큰은 태스크 의존 — 고정 휴리스틱 실패	Semantic Integrity Matters(2502.01941)	[3단계]
회귀·이른시점 로그가 진단적 (rare=cause)	Zebrium · Red Hat Cordon AetherLog(ISSRE'25)	[배경]
로그·RCA 특화 KV 압축 미개척 (novelty)	LLM4Log 서버이(2604.16359)	[3단계]
KV 메모리>가중치·처리량 급감 정량	FastKV §2.1 · Cartridges §1	[배경]

근거 신뢰도 주의: 핵심 3단계 논거는 동료심사 논문(H2O·Scissorhands·SnapKV NeurlPS, StreamingLLM·RazorAttention ICLR, Lost in the Middle TACL)에 기반. 'rare=진단적'은 일부 벤더 자료(Zebrium·Red Hat)라 보조 근거로만 사용. 미래 낱자 프리프린트(2604·2606)는 미심사라 가중치 하향. 표현력 논거(Cartridges)는 학습 기반 KV 한정이므로 eviction엔 head 단위 보존까지만 적용. 2단계 prefill 혼합 이점은 자체 ablation으로 재확인 권장.